



Bubble Blast

A 2D Android Arcade Shooter

Core Technologies: Java, Android SurfaceView, Thread-based Game Loop

Team: Aryan Chauhan, Savyam, Zaman



Project Vision



Physics-Based Shooting

Precision aiming using trigonometric calculations for realistic projectile motion



Multiple Weapon Types

Three distinct weapons offering different damage patterns and strategic options



Special Pulsar Ability

Screen-clearing shockwave power-up for intense survival situations

Arcade survival games demand exceptional performance to handle hundreds of moving objects simultaneously while maintaining smooth gameplay. Our objective was to create an engaging physics-based shooter that pushes the limits of mobile gaming performance.

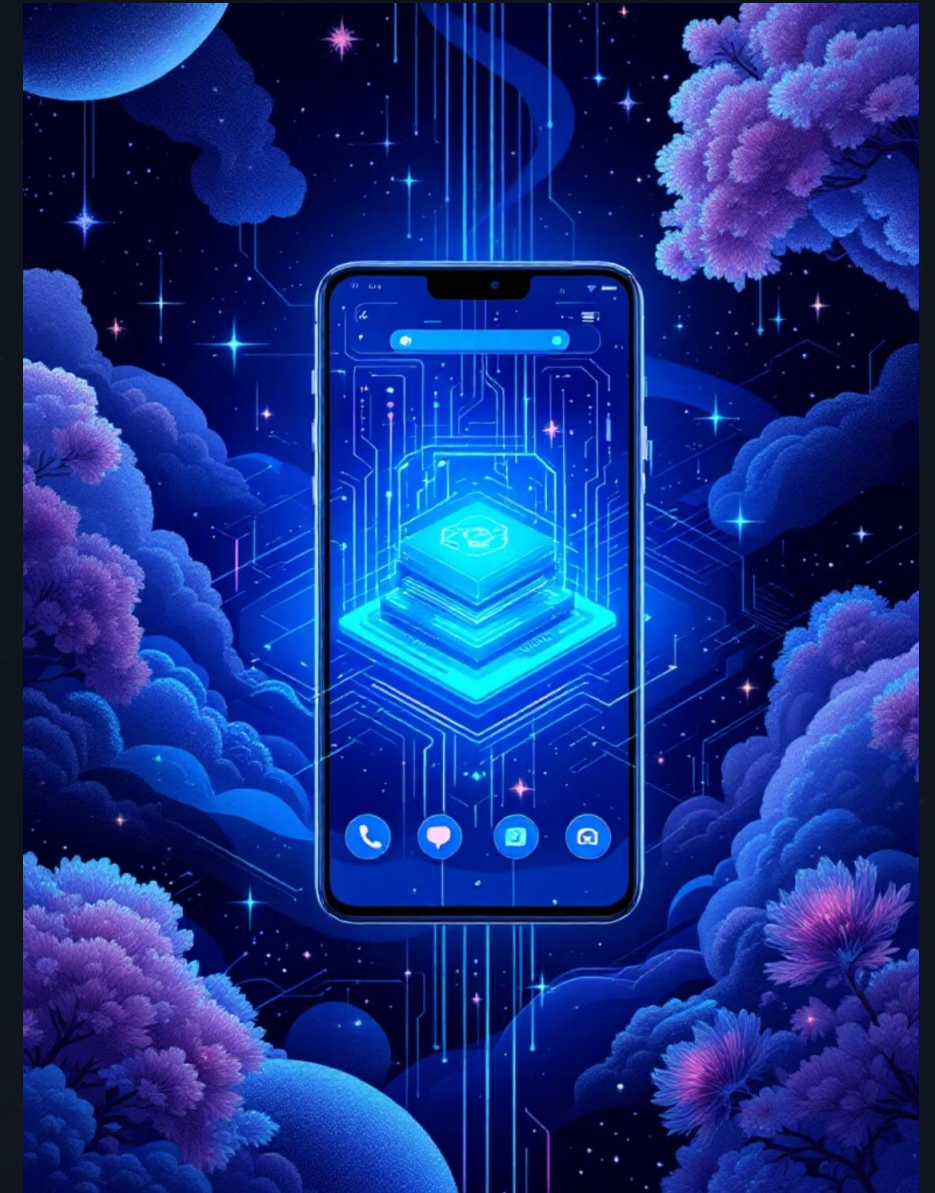


Problem Statement

Performance Challenges

Building a responsive arcade shooter required overcoming several technical hurdles that test the limits of Android platform capabilities.

- Managing 400+ simultaneous objects efficiently in real-time
- Maintaining consistent 60 FPS frame rate under heavy load
- Processing user input without perceptible lag
- Implementing real-time collision detection for dynamic objects



Technical Stack



Android Studio

Java JDK 17 with SurfaceView for accelerated rendering and hardware-accelerated graphics support



Trigonometry Engine

atan2, sin, and cos functions for precise aiming calculations and smooth object motion



Graphics Library

RadialGradient for bubble shading effects and LinearGradient for background depth and visual appeal



Core Game Architecture



Update Module

Game timing controller that moves bubbles and projectiles while managing screen effects



Collision Module

Euclidean distance formula implementation for precise overlap detection between objects



Particle Module

Smoke and explosion effects that add visual polish and feedback to gameplay



Weapon System

1

Bullet

Single target damage with fast projectile speed for precision shooting

2

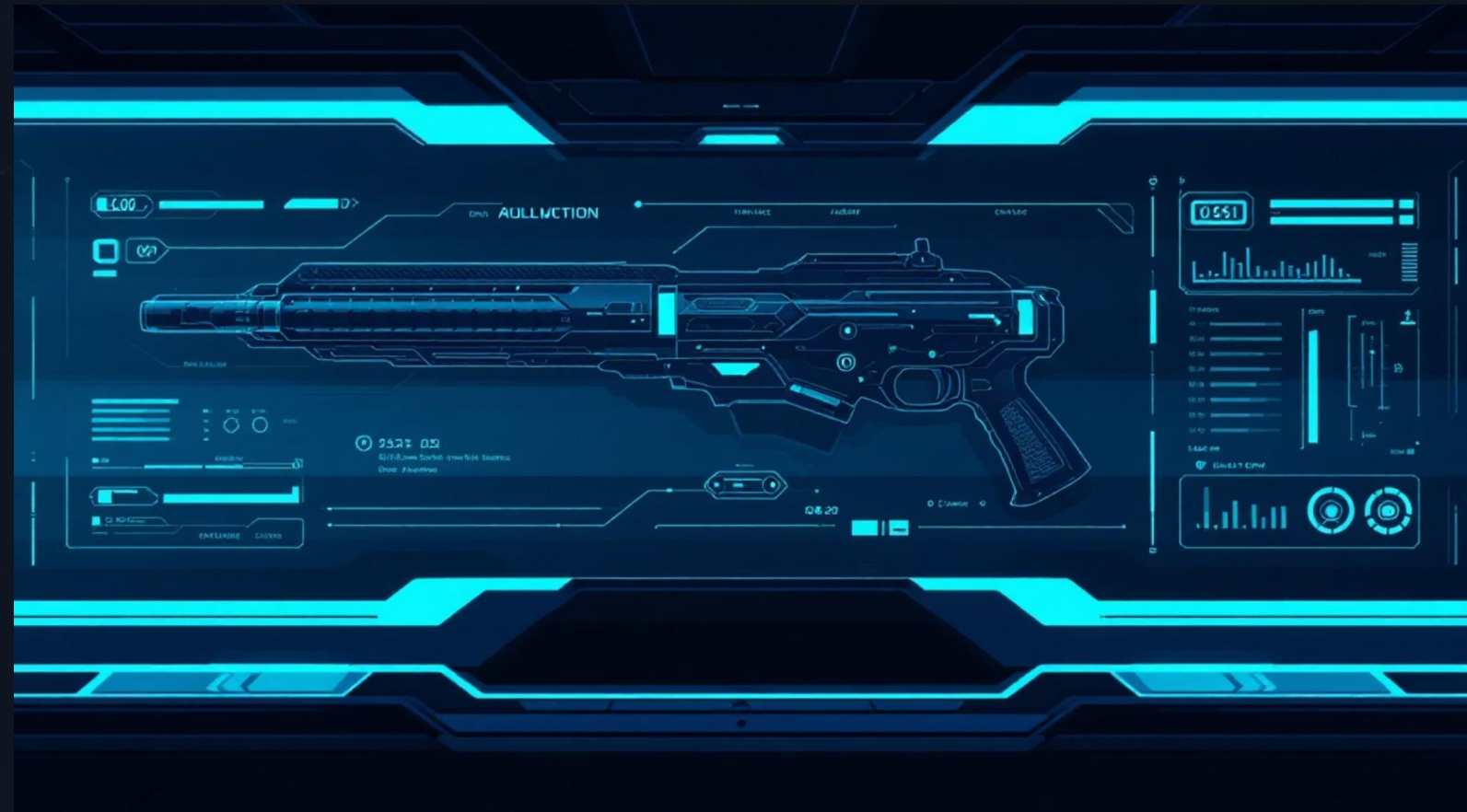
Grenade

Area damage within 250px radius for crowd control situations

3

Missile

Large explosion with 450px radius plus smoke effects for maximum impact



Special Ability

Pulsar System unleashes an expanding shockwave that clears all bubbles from the screen, providing a strategic advantage during intense moments.



Code Implementation Highlights

01

Dynamic Object Management

ArrayList efficiently manages 400+ bubbles with dynamic addition and removal during gameplay

02

Separate Game Thread

Dedicated thread ensures smooth 60 FPS rendering independent of UI thread operations

03

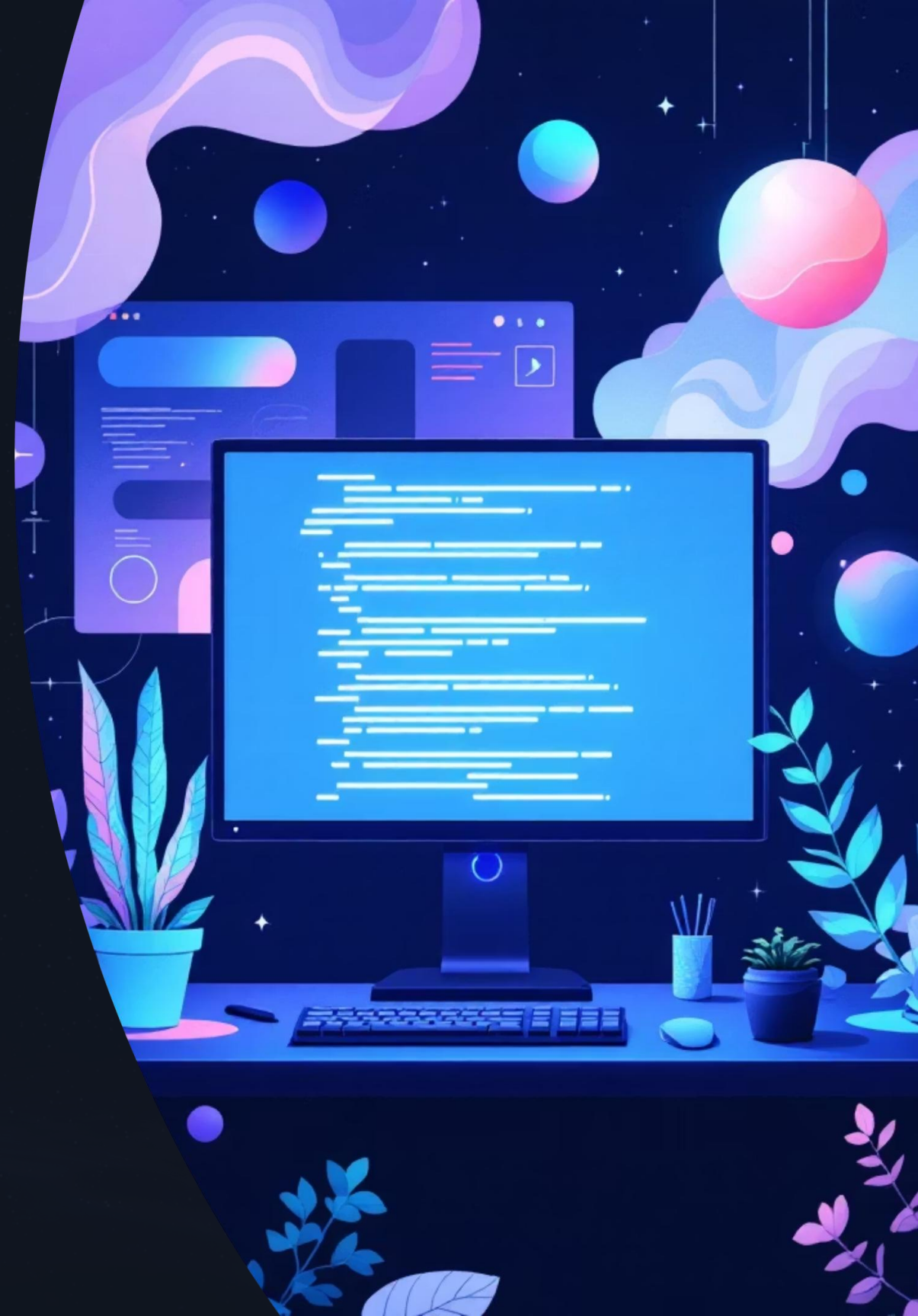
Real-Time Collision Detection

Continuous Euclidean distance calculations between all objects for accurate hit detection

04

Particle System Integration

Smoke and explosion effects enhance visual feedback while maintaining performance





Performance Metrics

60

Frames Per Second

Consistent rendering speed for
smooth gameplay

400+

Simultaneous Objects

Bubbles, projectiles, and particles
on screen

Optimization Techniques

- Object pooling to reduce garbage collection
- Spatial partitioning for collision optimization
- Hardware-accelerated SurfaceView rendering
- Thread-safe data structures for concurrent access



Development Team



Aryan Chauhan

Game architecture design and core physics implementation



Savyam

Graphics engine development and visual effects programming



Zaman

Weapon system design and collision detection algorithms



Key Takeaways



Scalable Architecture

Modular design allows easy expansion with new weapons and abilities



Performance First

Thread-based game loop and efficient collision detection ensure 60 FPS stability



Visual Polish

Gradient shading and particle effects create engaging arcade experience

Bubble Blast demonstrates how careful optimization and modern Android development techniques can deliver console-quality arcade experiences on mobile platforms. The combination of physics-based gameplay, strategic weapon selection, and special abilities creates an addictive survival challenge that pushes hardware limits while maintaining buttery-smooth performance.

